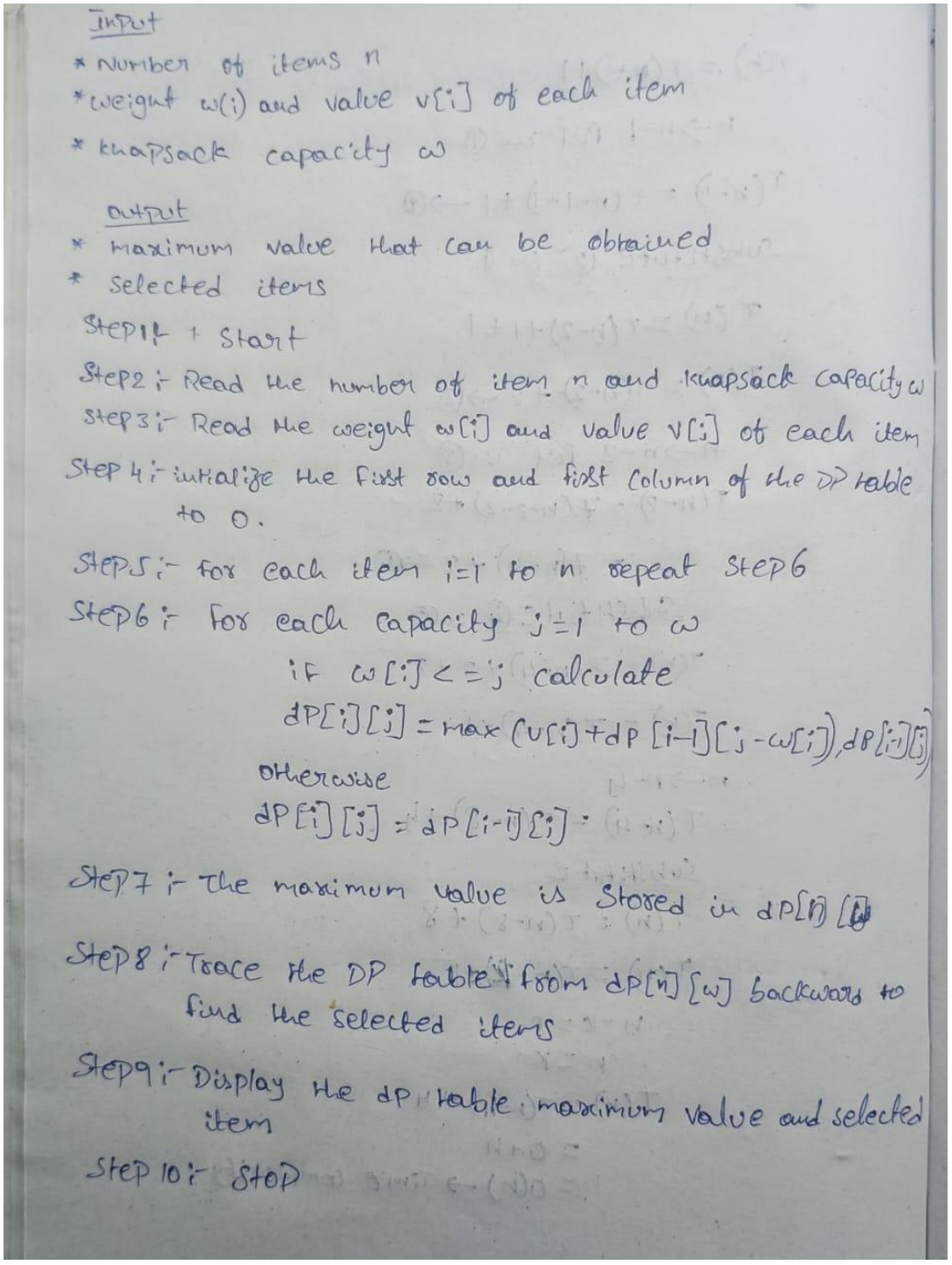
# Practical- 5

**Aim:** To Implementation of a knapsack problem using dynamic programming.

**Knapsack**

**Algorithm:**



Input
* Number of items n
* weight w(i) and value v[i] of each item
* knapsack capacity w

output
* maximum value that can be obtained
* selected items

Step1 : Start
Step2 : Read the number of item n and knapsack capacity w
Step3 : Read the weight w[i] and value v[i] of each item
Step 4 : initialize the first row and first column of the DP table to 0.
Step5 : for each item i=1 to n repeat step6
Step6 : for each capacity j=1 to w
if w[i] <= j calculate
dP[i][j] = max (v[i] + dP [i-1][j - w[i]), dP[i-1][j])
otherwise
dP[i][j] = dP[i-1][j]
Step7 : The maximum value is stored in dP[n][w]
Step8 : Trace the DP table from dP[n][w] backwards to find the selected items
Step9 : Display the dP table, maximum value and selected item
Step 10 : Stop

Name: Y.Sudheer Kumar
Enrollment Number:92400118922

**Program:**

```
#include<iostream>
using namespace std;
class Knapsack {
private:
   int w[10];
   int v[10];
   int n;
   int W;
   int dp[10][100];
public:
   void getInput() {
      cout << "Enter number of items  : ";
      cin >> n;
      cout << "Enter knapsack capacity : ";
      cin >> W;
      for(int i = 1; i <= n; i++) {
         cout << "Item " << i << " weight and value : ";
         cin >> w[i] >> v[i];
      }
   }
   void solve() {
      for(int i = 0; i <= n; i++) dp[i][0] = 0;
      for(int j = 0; j <= W; j++) dp[0][j] = 0;
      for(int i = 1; i <= n; i++) {
         for(int j = 1; j <= W; j++) {
            if(w[i] <= j) {
               dp[i][j] = max(
                  v[i] + dp[i-1][j-w[i]],
                  dp[i-1][j]
               );
            } else {
               dp[i][j] = dp[i-1][j];
```

```
            }
        }
    }
}
void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "     ";
    for(int j = 0; j <= W; j++)
        cout << j << "  ";
    cout << endl;
    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << "  ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << "  ";
        cout << endl;
    }
}
void findItems() {
    cout << "\nSelected Items:" << endl;
    int j = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
                 << ") TAKEN " << endl;
            j = j - w[i];
        } else {
            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
                 << ") NOT taken " << endl;
        }
```

```
        }
    }
    void showResult() {
        cout << "\nMaximum Value = "
             << dp[n][W] << endl;
    }
};
int main() {
    Knapsack k;
    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();
    return 0;
}
```

**Output:**

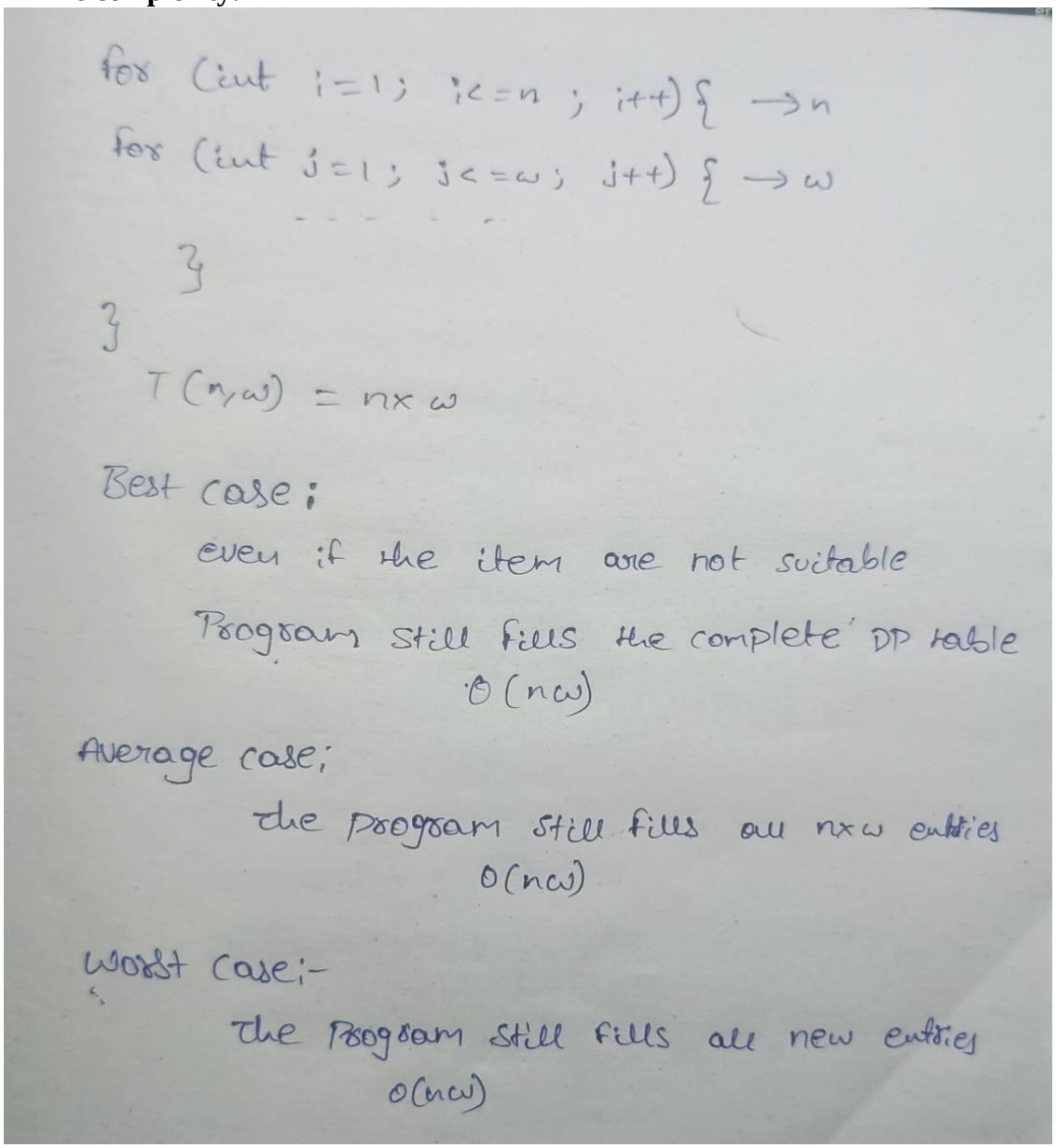
```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\practical-5\knapsnack.exe"
Enter number of items   : 4
Enter knapsack capacity : 8
Item 1 weight and value : 2
1
Item 2 weight and value : 3
2
Item 3 weight and value : 4
5
Item 4 weight and value : 5
6

DP Table:
     0  1  2  3  4  5  6  7  8
i=0  0  0  0  0  0  0  0  0  0
i=1  0  0  1  1  1  1  1  1  1
i=2  0  0  1  2  2  3  3  3  3
i=3  0  0  1  2  5  5  6  7  7
i=4  0  0  1  2  5  6  6  7  8

Maximum Value = 8

Selected Items:
Item 4 (w=5, v=6) TAKEN
Item 3 (w=4, v=5) NOT taken
Item 2 (w=3, v=2) TAKEN
Item 1 (w=2, v=1) NOT taken
PS C:\MinGW\bin>
```

**Time complexity:**




**Conclusion:** The 0/1 Knapsack Problem was successfully implemented using Dynamic Programming. The algorithm efficiently finds the maximum value that can be obtained within the given knapsack capacity by considering whether each item should be selected or not selected. The time complexity of the algorithm is O(nW) and the space complexity is O(nW).

Name: Y.Sudheer Kumar
Enrollment Number:92400118922